

Security Assessment

Workspace (BuildPact) Program

August 24, 2026

Overview

Project Summary

Project Name	Workspace (BuildPact)
Language	Rust (Anchor Framework)

Audit Summary

Delivery Date	August 24, 2026 UTC
Audit Methodology	Static Analysis, Manual Review

Vulnerability Summary

Vulnerability Level	Total
Critical	0
Major	0
Medium	2
Minor	2

Table of Contents

Summary

Overview

Project Summary

Audit Summary

Vulnerability Summary

Audit Scope (Scope includes all provided Rust files)

Findings

Finding-01 : Unused Global Configuration and Dead Code

Finding-02 : Locked Funds Due to Inability to Close Finalized Projects

Finding-03 : String Length Validation Inconsistencies

Finding-04 : Missing On-Chain Event Emissions

Findings

4 Total Issues

Major: 0 (0%), Medium: 2 (50%), Minor: 2 (50%)

Title	Category	Severity
Unused Global Configuration and Dead Code	Architectural Flaw	Medium
Locked Funds Due to Inability to Close Finalized Projects	Logical Issue	Medium
String Length Validation Inconsistencies	Data Validation	Minor
Missing On-Chain Event Emissions	Best Practices	Minor

Unused Global Configuration and Dead Code

Category	Architectural Flaw
Severity	Medium

Description

The program includes an instruction to initialize a configuration account, which sets up global protocol parameters such as the protocol fee percentage, protocol status (paused or active), and authority. However, this configuration account is completely isolated from the rest of the program logic.

Core instructions like project creation and fund distribution do not require the configuration account to be passed in, nor do they read its values. Instead, they rely on hardcoded constants defined at the top of the file for the protocol wallet and protocol fee. This results in the configuration initialization being dead code, misleading users or administrators into believing they can dynamically update fees or pause the contract, when in reality, they cannot without upgrading the entire program.

Recommendation

It is recommended to evaluate the long-term architectural goals of the protocol. If dynamic configuration (such as pausing the protocol in emergencies or updating the fee structure) is required, the configuration account must be integrated into all relevant instructions. The program should mandate passing this configuration account and verifying its values rather than relying on hardcoded constants. Conversely, if static constants are preferred for immutability and gas efficiency, the initialization instruction and the configuration struct should be completely removed to clean up dead code and avoid confusion.

Locked Funds Due to Inability to Close Finalized Projects

Category	Logical Issue
Severity	Medium

Description

The program provides an instruction to close a project, which successfully refunds the lamports held in the project's vault to the creator and closes the project account. However, this instruction strictly requires the project status to be "Open". Once a project is finalized, it can never be closed.

While preventing the closure of an active, finalized project protects members' incoming funds, it introduces a problem after a project has completed its lifecycle. During fund distribution, integer division rounding down inherently leaves behind tiny fractions of lamports (dust) in the vault. Because finalized projects cannot be closed, this accumulated dust, along with the standard rent-exempt minimum balance held by the vault and the project state account, becomes permanently locked in the contract forever.

Recommendation

It is recommended to introduce a mechanism to fully conclude or archive a project after all milestones or financial obligations are met. This could involve adding a new status such as "Completed", which members must mutually approve, or introducing a time-based unlock. Once a project reaches this completed state, an authorized user (such as the creator) should be permitted to close the project accounts, allowing them to recover the rent exemption and any residual lamport dust, thereby preventing unnecessary state bloat on the Solana network.

String Length Validation Inconsistencies

Category	Data Validation
Severity	Minor

Description

When creating a project or adding a member, the program enforces maximum length limits on various text fields (such as titles, descriptions, and roles) to ensure they fit within the predefined account space. It checks these limits using the standard Rust length method on strings.

However, this standard method returns the length of the string in bytes, not characters. Because Solana utilizes UTF-8 encoding, characters such as emojis, accented letters, or non-Latin alphabets take up multiple bytes. Consequently, a user could submit a string that is well within the acceptable character limit but gets rejected by the program because its multi-byte characters exceed the predefined byte limit.

Recommendation

It is recommended to ensure that frontend interfaces explicitly validate byte length rather than character length before submitting transactions to the network. Clear documentation should be provided to users and integrators regarding these limitations. For the smart contract, keeping byte-length checks is generally preferred over character counting due to computational efficiency, but the discrepancy should be acknowledged and managed at the client application layer.

Missing On-Chain Event Emissions

Category	Best Practices
Severity	Minor

Description

The program handles complex state transitions involving multiple participants, such as initializing projects, updating member rosters, capturing approvals, and processing fund distributions. Despite these significant interactions, the contract does not emit any on-chain events.

Without standard event emissions, off-chain infrastructure, indexing services, and client frontends must resort to complex transaction log parsing or constant account polling to determine when a project's status changes or when a user receives funds. This makes the overall system less robust and harder to integrate with external applications.

Recommendation

It is recommended to define and implement Anchor events for all critical state changes within the program. Events should ideally be emitted when a project is created, when members are added or removed, when a member approves their role, when the project status officially changes to finalized, and every time funds are distributed to participants.

Summary

This report summarizes the security assessment of the Workspace (BuildPact) Solana Program written in Rust using the Anchor framework. The assessment involved a comprehensive examination of the provided source code using Static Analysis and Manual Review techniques.

The audit focused on identifying potential vulnerabilities, logical flaws, and adherence to Solana and Anchor best practices. Key areas of focus included:

- Account validation and access control mechanics.
- Handling of native token transfers and Cross-Program Invocations (CPI).
- Mathematical precision, checked operations, and fee calculations.
- State management, bounds checking, and data integrity.

The assessment found the mathematical distribution logic to be solid, appropriately utilizing checked math to prevent overflow and effectively validating percentage logic (Basis Points). Access controls around member approvals and project finalization are properly implemented.

However, the review identified two Medium severity issues related to overall architectural planning. The presence of an unused configuration account indicates a partial shift in design paradigms that was not fully cleaned up. Furthermore, the inability to close fully executed projects results in permanently locked lamports (both in rent and mathematical dust). Addressing these lifecycle and architectural items, alongside minor data validation and best-practice improvements, will highly

Disclaimer

For the purposes of this disclaimer:

- "*Noah AI*" refers to the product currently operated by its owners and developers, including but not limited to its creators, contributors, affiliates, officers, employees, contractors, and agents.
 - "*Plena*" refers to Plena Finance Digital LLC, including but not limited to its affiliates, officers, employees, contractors, and agents.
 - "*User*" refers to the recipient of this report, whether an individual or an entity, and any of its affiliates, officers, employees, contractors, or agents who access, review, or act upon the contents of this report.
-

AI-Generated Content

This report is *entirely generated by artificial intelligence (AI)* without human authorship, review, or verification by Noah AI or Plena. AI-generated content is inherently probabilistic and may contain inaccuracies, false positives, or false negatives. *The User is solely and fully responsible* for reviewing, verifying, and validating all information before relying on it or taking any action.

No Responsibility or Liability

To the maximum extent permitted by law, *Noah AI and Plena, and their respective owners, operators, affiliates, officers, employees, contractors, and agents*, **accept no responsibility or liability** for:

- The accuracy, completeness, legality, reliability, or security of this report
 - Any hacks, exploits, unauthorized access, security breaches, or vulnerabilities
 - Any loss of funds, data, or digital assets
 - Any direct, indirect, incidental, consequential, punitive, or special damages of any kind, *whether in contract, tort, or otherwise*, even if advised of the possibility of such damages
-

No Endorsement or Advice

Nothing in this report constitutes or should be interpreted as an endorsement, recommendation, disapproval, investment advice, financial advice, legal advice, or guarantee of performance.

No Guarantee of Regulatory Compliance

Nothing in this report constitutes a guarantee by Noah AI or Plena of regulatory compliance, legal validity, or adherence to applicable laws in any jurisdiction.

No Contract Formation

Use of this report does not create any contractual obligations between the User and Noah AI, Plena, or their respective owners, beyond those explicitly stated in a written agreement between the parties.

Third-Party Dependencies

References to third-party technologies, services, or platforms are for informational purposes only and do not constitute endorsements or assurances by Noah AI or Plena of their reliability, security, or legality.

Risk Disclaimer

Blockchain technology, AI systems, and cryptographic assets are highly experimental and carry substantial risks. The User remains solely responsible for conducting independent due diligence, obtaining professional advice, and ensuring compliance with all applicable laws and regulations before acting on any information in this report.